

Transpiling Rust to SMT: A Framework for Formal Verification

MEHRAD HAGHSHENAS, University of Waterloo, Canada

Formal verification provides assurance of the correctness of programs by constructing proofs of logical propositions which describe the behavior of the program. In recent years, significant efforts have been made to develop software tools that construct proofs. These software generally fall under two categories: *Automated Theorem Provers* and *Proof Assistants*. The former, automated theorem provers, such as SMT solvers (e.g., Z3), provide a *push-button* experience. In other words, a logical proposition is submitted to the tool, and the tool returns whether it is satisfiable. Accordingly, software verification using SMT solvers involves three main steps:

- (1) First, transpiling the program into a language recognized by the SMT solver.
- (2) Second, transpiling the specifications and properties into a language recognized by the SMT solver.
- (3) In the SMT solver, proving that the program satisfies the specifications.

Although automated theorem provers have improved a lot over the recent years, a significant limitation for the adoption of formal verification is the transpiling phase. This project proposes *Rusmart*, a subset of the Rust language augmented with logical annotations to facilitate automatic formal verification using SMT solvers. Specifically, *Rusmart* aims to enable writing of operational and denotational semantics together. Therefore, the code, operational semantics, and specifications, denotational semantics, can be written and given to SMT for constructing the proof. The novelty of this approach arises from directly embedding SMT semantics into Rust, rather than relying on external verification scripts.

1 Introduction

Rusmart is presented as a subset of Rust enhanced with annotations to bridge the gap with SMT (Satisfiability Modulo Theories) solvers. Each element, i.e., function or type in Rusmart, should be marked with one and *only* one of the following annotations: `#[smt_type]` for types, `#[smt_impl]` for implementations, `#[smt_spec]` for specifications and `#[smt_axiom]` for axioms. The implementations are what the function is expected to do, or in other words describes the *operational semantics* of the code. `#[smt_impl]` can have attributes which define the specification to which it relates. Furthermore, the specifications are the *denotational semantics*, which can be defined explicitly or left unimplemented. In the case that it is unimplemented (that is we do not know what the code precisely does), the axioms bridge the connection between what is expected from the code and what the code actually does. Similar to `#[smt_impl]`, `#[smt_spec]` can have attributes declaring the implementation it specifies. With regards to type, each user-defined type, *enum* or *struct*, must be marked with an annotation `#[smt_type]` which cannot have any attributes. The `#[smt_axiom]` annotation, which does not permit any attributes, is used to declare a function as an axiom. To emphasize, the `#[smt_axiom]`, `#[smt_impl]`, and `#[smt_spec]` notations are only used before function declarations and the `#[smt_type]` is used before type declarations. The following sections detail the expressions, types, and functions supported by Rusmart, highlighting its expressiveness.

2 Types

Rusmart is a subset of Rust primarily developed for formal verification. In rustmart, only types that are defined in SMT solvers such as Z3 are allowed. Therefore, the allowed types are as following: *Integer*, *Boolean*, *Rational*, *Text*, *Error*, *Cloak*, *Seq*, *Set*, & *Array*. Specifically, *Boolean* captures boolean logic, *Integer* & *Rational* offer arbitrary-precision numeric types, and *Text* wraps a Rust string. Collection types include *Seq* for ordered sequences, *Set* for unordered collections, and *Map* for

key-value pairs. Finally, Cloak is a specialized wrapper that enables recursive data definitions by preventing cycles. Along with these data types, Rusmart provides quantified expressions, such as *forall*, *exist*, and *choose* - allowing universal, existential, and choice-based logic to be natively expressed.

3 Expressions

Every function in Rusmart must have a return type. axioms must return a Boolean value and the type signature of corresponding specifications and implementations must match. Inside the body of an smt-annotated function let-bindings are allowed. For a list of allowed statements and expressions please look at the *expr.rs* module of the *parser* crate.

4 Organization

A Rusmart program is in essence a **.rs* file written in the Rusmart language which is a subset of Rust. The *rusmart-smt-remark* crate will enrich the abstract syntax tree (AST) of the Rusmart program. The *rusmart-smt-derive* crate derives the denotational semantics of the program. *rusmart-smt-backend-parser* parses the program, *rusmart-smt-backend-ir* creates the intermediate representation, and *rusmart-smt-backend-backend* generates the Z3 smt2 code.

5 Roadmap and Milestones

At the moment the whole pipeline is constructed, that is, example Rusmart programs can be written and the verification of these programs can be automatically proven. Nevertheless, the following objectives remain to be implemented:

- (1) By writing tests, some bugs have been found. The main objective will be to first resolve the bugs and write more tests for a more complete coverage.
- (2) For *Iterforall*, *Existsforall*, and *Chooseforall* expressions the smt generation is incomplete.
- (3) The language's current expressiveness is limited. For example, it cannot support a sorting algorithm. We plan to review and integrate the full range of functionalities offered by Z3 to enhance Rusmart's compatibility.

The primary objective is to finalize the software so that it can be used to model a variety of example languages. This will allow proving properties of these languages.